

UNIVERSITÀ DEGLI STUDI DI UDINE

Corso di Internet of Things



MagicHands

*Interfaccia gestuale contactless con autenticazione biometrica su
dispositivo embedded*

STUDENTE

Roland Gjopalaj

Matricola: 157277

DOCENTE

Prof. Ivan Scagnetto

Anno Accademico 2024/2025

1. Descrizione del Progetto

MagicHands è un sistema IoT di controllo gestuale autenticato, progettato per essere eseguito su Raspberry Pi 4. Il sistema integra il riconoscimento facciale e la rilevazione dei gesti della mano in un'unica pipeline, consentendo a utenti autorizzati di controllare la riproduzione musicale e il volume attraverso semplici gesti delle dita, senza alcun contatto fisico con il dispositivo.

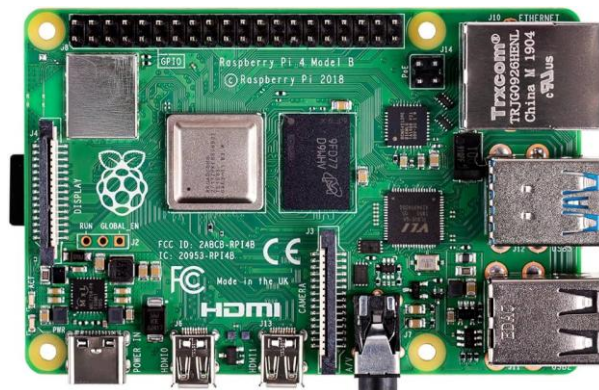
L'idea centrale del progetto è quella di combinare due tecnologie di computer vision, face recognition e hand gesture detection per creare un'interfaccia uomo-macchina intuitiva e sicura, con feedback visivo tramite un LED collegato ai pin GPIO del Raspberry Pi.

2. Componenti Hardware

Il sistema è realizzato con componenti di facile reperibilità e basso costo. Di seguito sono elencati i componenti principali utilizzati.

2.1 Raspberry Pi 4

Il Raspberry Pi 4 Model B costituisce il cuore computazionale del sistema. Grazie alla sua CPU quad-core ARM Cortex-A72 a 64 bit e alla memoria RAM (4 GB nella configurazione utilizzata), è in grado di gestire in tempo reale i carichi computazionali richiesti dai modelli di visione artificiale.



2.2 Picamera2 (Camera Module)

La videocamera è gestita tramite la libreria Picamera2, l'interfaccia ufficiale per i moduli camera di Raspberry Pi. Il sistema utilizza due configurazioni distinte della stessa camera, ottimizzate per i due task:

- Riconoscimento facciale: risoluzione 1920×1080 per catturare dettagli facciali precisi
- Rilevazione gesti: risoluzione ridotta 640×480 per alta frequenza di acquisizione e bassa latenza



2.3 LED + Resistenza (GPIO)

Un LED verde è collegato al pin GPIO 26 del Raspberry Pi tramite una resistenza limitatrice di corrente. Il LED funge da indicatore visivo di stato: si accende quando un utente autorizzato viene riconosciuto e la finestra gestuale è attiva, e si spegne automaticamente al termine dei 5 secondi.



3. Architettura Software

3.1 Panoramica delle Librerie Utilizzate

Il progetto si basa su diverse librerie Python specializzate per la visione artificiale e il controllo hardware:

Libreria	Funzione	Note
<code>face_recognition</code>	Riconoscimento facciale	Basata su dlib; confronto tramite distanza euclidea tra encoding
<code>OpenCV (cv2)</code>	Elaborazione immagini	Ridimensionamento frame, conversione spazi colore
<code>MediaPipe Hands</code>	Rilevazione gesti mano	Modello ML di Google per il tracking dei 21 landmark della mano
<code>Picamera2</code>	Interfaccia camera	Driver ufficiale per i moduli camera Raspberry Pi
<code>gpiozero</code>	Controllo GPIO	Gestione LED sul pin GPIO 26
<code>subprocess</code>	Controllo sistema	Esecuzione comandi amixer e playerctl
<code>threading</code>	Concorrenza	Usato nella versione multi-thread del progetto
<code>pickle</code>	Serializzazione	Salvataggio/caricamento degli encoding facciali

3.2 Pipeline di Funzionamento

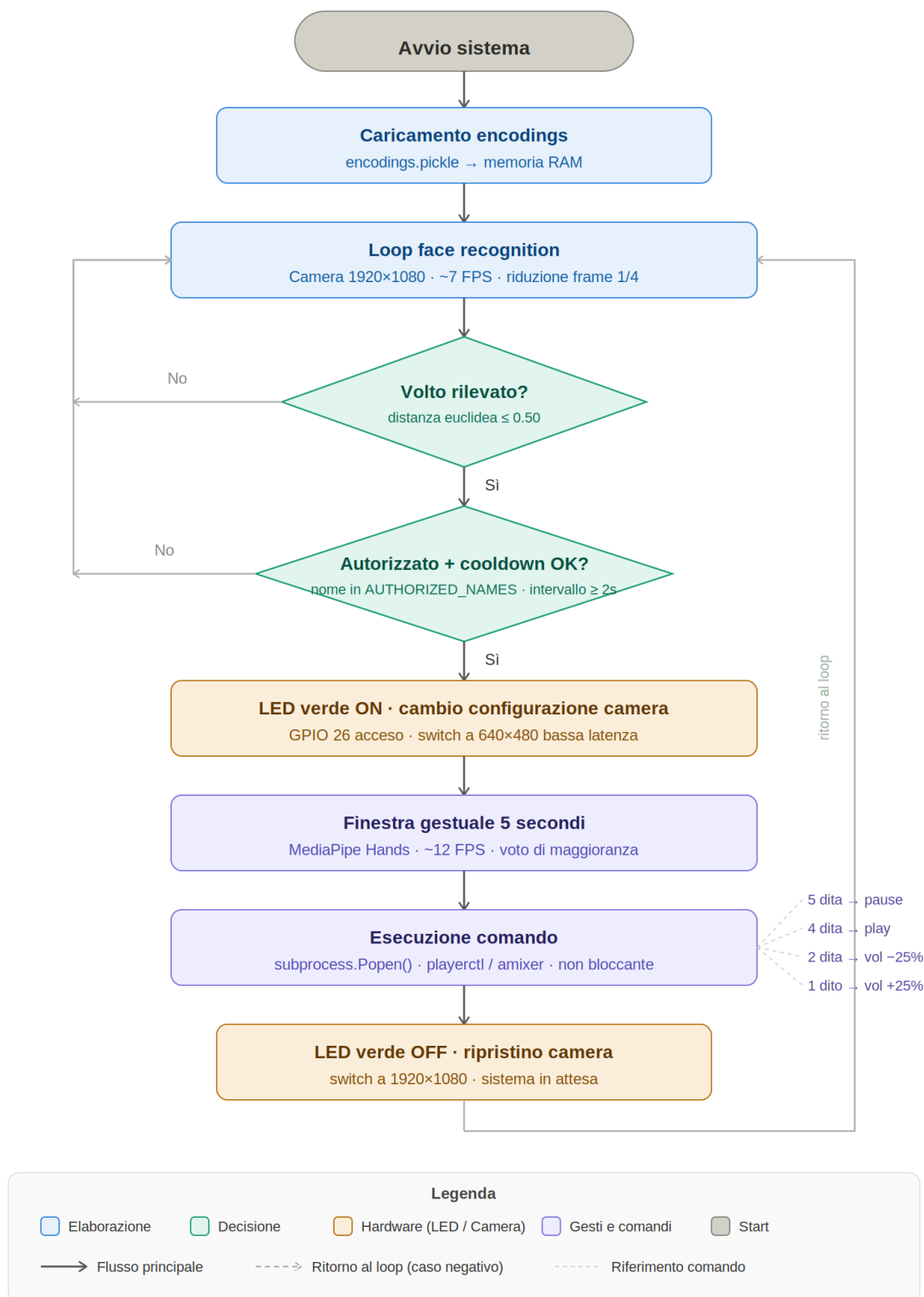
Il sistema opera seguendo una pipeline sequenziale ben definita, che può essere riassunta nei seguenti passaggi:

- **Fase 1 – Acquisizione:** La camera acquisisce continuamente frame in alta risoluzione (1920×1080) durante la modalità di sorveglianza facciale.
- **Fase 2 – Pre-elaborazione:** Ogni frame viene ridimensionato di un fattore 4 (a 480×270) prima di essere passato al modello HOG per la face detection, riducendo significativamente il carico computazionale.
- **Fase 3 – Riconoscimento facciale:** Il modello face_recognition calcola un encoding a 128 dimensioni del volto rilevato e lo confronta con gli encoding degli utenti autorizzati tramite distanza euclidea. Se la distanza è inferiore alla soglia configurata (FACE_TOLERANCE = 0.50), l'utente viene considerato autorizzato.
- **Fase 4 – Attivazione:** In caso di riconoscimento positivo, il LED GPIO si accende e si apre una finestra temporale di 5 secondi. La camera viene riconfigurata a bassa risoluzione (640×480) per massimizzare il framerate.
- **Fase 5 – Gesture Detection:** MediaPipe Hands analizza i landmark della mano (21 punti per mano) e conta le dita alzate. I campioni vengono raccolti per tutta la durata della finestra e la scelta finale è determinata dalla moda (valore più frequente).
- **Fase 6 – Esecuzione Comando:** In base al numero di dita rilevato viene eseguito il comando corrispondente tramite subprocess. Al termine, il LED si spegne e la camera torna alla configurazione per il face recognition.

```
[FACE] Volto autorizzato: andi - avvio thread DJ.
[DJ] LED acceso - finestra comandi attiva.

[GESTURE] Cambio configurazione camera → 640x480 bassa latenza...
[0:05:56.717320893] [1897] INFO Camera camera.cpp:1215 configuring streams: (0) 640x480-XRGB8
888/sRGB (1) 640x480-SGBRG10_CSI2P/RAW
[0:05:56.722407074] [1884] INFO RPI vc4.cpp:615 Sensor: /base/soc/i2c0mux/i2c@1/ov5647@36 - S
elected sensor format: 640x480-SGBRG10_1X10/RAW - Selected unicam format: 640x480-pGAA/RAW
[GESTURE] Mostra la mano! Hai 5s per il comando.
W0000 00:00:1774112205.268675 1892 landmark_projection_calculator.cc:186] Using NORM_RECT w
ithout IMAGE_DIMENSIONS is only supported for the square ROI. Provide IMAGE_DIMENSIONS or use
PROJECTION_MATRIX.
[ ] 0.0s Dita rilevate: 4
[GESTURE] Campioni: [1, 1, 1, 1, 1, 1, 5, 5, 4, 4, 5, 5, 5, 4, 4, 4, 4, 4, 4, 4]
→ Scelta finale: 4 dita
[CMD] Eseguo comando per 4 dita: playerctl play
[GESTURE] Ripristino configurazione camera → 1920x1080 face recognition...
[0:06:02.208628111] [1897] INFO Camera camera.cpp:1215 configuring streams: (0) 1920x1080-XRG
B888/sRGB (1) 1920x1080-SGBRG10_CSI2P/RAW
[0:06:02.210634079] [1884] INFO RPI vc4.cpp:615 Sensor: /base/soc/i2c0mux/i2c@1/ov5647@36 - S
elected sensor format: 1920x1080-SGBRG10_1X10/RAW - Selected unicam format: 1920x1080-pGAA/RAW
No players found
[DJ] LED spento - sistema in attesa.
```

Schema a Blocchi — Flusso Operativo MagicHands



4. Riconoscimento Facciale

4.1 Fase di Training: Acquisizione Immagini

Prima dell'utilizzo del sistema, è necessario popolare il dataset con le immagini degli utenti autorizzati. Il file `image_capture.py` gestisce questa fase: all'avvio chiede il nome dell'utente, inizializza la camera e mostra un'anteprima live. Premendo la barra spaziatrice l'utente acquisisce una fotografia, che viene salvata nella cartella `dataset/<nome>/` con un timestamp per evitare conflitti.

Si raccomanda di acquisire almeno 10-20 immagini per utente, variando l'angolazione e le condizioni di illuminazione, per garantire robustezza al modello.

4.2 Fase di Training: Generazione degli Encoding

Il file `model_training.py` processa tutte le immagini nel dataset e genera gli encoding facciali. Per ogni immagine:

- Individua i volti tramite il modello HOG (Histogram of Oriented Gradients)
- Calcola un vettore di 128 valori numerici (encoding) che rappresenta in modo univoco il volto
- Associa l'encoding al nome della persona (ricavato dal nome della cartella)

Gli encoding vengono serializzati in un file `encodings.pickle`, che viene caricato all'avvio del sistema principale.

4.3 Riconoscimento in Tempo Reale

Durante il funzionamento, `process_frame()` esegue il confronto tra l'encoding del volto rilevato e quelli nel database usando `face_distance()`, che calcola la distanza euclidea. Il candidato con distanza minima viene scelto come match. Se la distanza è ≤ 0.50 (parametro `FACE_TOLERANCE`) e il nome è nella lista `AUTHORIZED_NAMES`, l'utente viene autenticato.

5. Rilevazione dei Gestii

5.1 Modello MediaPipe Hands

MediaPipe Hands è un modello di machine learning sviluppato da Google che rileva e traccia la mano identificando 21 landmark tridimensionali.



Il sistema utilizza i seguenti landmark per determinare lo stato delle dita:

Dito	Tip (punta)	PIP / DIP	Logica di rilevazione
Pollice	4	2 (PIP)	Confronto asse X
Indice	8	6 (PIP)	$\text{Tip.y} < \text{PIP.y} \rightarrow$ dito alzato
Medio	12	10 (PIP)	$\text{Tip.y} < \text{PIP.y} \rightarrow$ dito alzato
Anulare	16	14 (PIP)	$\text{Tip.y} < \text{PIP.y} \rightarrow$ dito alzato
Mignolo	20	18 (PIP)	$\text{Tip.y} < \text{PIP.y} \rightarrow$ dito alzato

Nel sistema di coordinate di un'immagine, l'asse Y è **invertito** rispetto all'intuizione: lo 0 è in alto e i valori aumentano verso il basso.



5.2 Mappatura Gestì → Comandi

Il numero di dita alzate viene mappato ai seguenti comandi di sistema:

Dita	Azione	Comando Sistema	Strumento
1 🖐️	Aumenta volume +25%	<code>amixer sset Master 25%+</code>	ALSA
2 🖐️	Diminuisce volume -25%	<code>amixer sset Master 25%-</code>	ALSA
4 🖐️	Avvia la musica	<code>playerctl play</code>	playerctl
5 🖐️	Mette in pausa	<code>playerctl pause</code>	playerctl

Il valore finale viene determinato tramite la moda statistica dei campioni raccolti durante la finestra temporale, garantendo robustezza rispetto a rilevazioni errate nei singoli frame.

6. Le Due Versioni del Sistema

Il progetto è stato sviluppato in due versioni distinte, che differiscono per il modello di esecuzione adottato: una versione sequenziale e una versione con threading. Entrambe condividono la stessa logica, ma si differenziano significativamente per le caratteristiche architetturelle e di performance.

6.1 Versione Sequenziale (sequenzial-code.py)

Nella versione sequenziale, le operazioni di face recognition e gesture detection vengono eseguite in modo strettamente seriale all'interno di un unico thread di esecuzione. Il flusso è il seguente:

- Il loop principale acquisisce un frame e tenta il riconoscimento facciale
- Se viene rilevato un utente autorizzato, accende il LED e chiama `gesture_detection()`
- `gesture_detection()` blocca l'esecuzione principale per 5 secondi, raccoglie i campioni e poi restituisce il controllo
- Solo dopo la fine della gesture detection il loop riprende a cercare volti

Questa versione è più semplice da comprendere e debuggare, ma ha il limite che durante la finestra gestuale il sistema non è in grado di rilevare nuovi volti. L'istanza di MediaPipe Hands viene creata e distrutta ad ogni attivazione, e la camera viene stop/start ad ogni cambio di configurazione.

6.2 Versione con Threading (magic_hands.py)

La versione avanzata introduce il multi-threading per disaccoppiare il loop di face recognition dall'esecuzione della gesture detection. Le principali ottimizzazioni architetturelle sono:

- **Thread separato per `activate_dj()`:** Quando viene rilevato un volto autorizzato, la gestione del LED e la gesture detection vengono avviati in un thread daemon separato, permettendo al loop principale di continuare a girare
- **Lock di concorrenza:** Un `threading.Lock()` protetto da variabile booleana (`is_dj_flag`) evita che attivazioni concorrenti si sovrappongano
- **Cooldown tra attivazioni:** La variabile `last_activation` implementa un cooldown di 2 secondi (`FACE_COOLDOWN`) per evitare attivazioni multiple consecutive sullo stesso volto
- **Istanza unica di MediaPipe:** L'oggetto `hands_model` viene creato una sola volta all'avvio e riutilizzato, eliminando il costo di inizializzazione del modello ad ogni attivazione
- **`subprocess.Popen()` vs `subprocess.run()`:** I comandi di sistema vengono eseguiti in modo non bloccante tramite `Popen()`, evitando attese durante l'esecuzione dei comandi audio

6.3 Confronto tra le Due Versioni

Caratteristica	Sequenziale	Threading
Modello esecuzione	Singolo thread	Multi-thread (daemon)
Face recog. durante gesture	Bloccato	Attivo in parallelo
Istanza MediaPipe	Creata ad ogni attivazione	Unica, riutilizzata
Comandi subprocess	Bloccante (run)	Non bloccante (Popen)
Cooldown tra attivazioni	Non implementato	2 secondi (FACE_COOLDOWN)
Complessità del codice	Bassa	Media
Robustezza	Sufficiente	Alta

7. Parametri di Configurazione

Il sistema è facilmente configurabile tramite le costanti definite nella sezione CONFIGURAZIONE in cima al file principale. I parametri principali sono:

Parametro	Valore default	Descrizione
<code>AUTHORIZED_NAMES</code>	<code>["andi", "peppe"]</code>	Lista dei nomi autorizzati al sistema
<code>GPIO_PIN</code>	26	Pin GPIO a cui è collegato il LED
<code>GESTURE_DURATION</code>	5 s	Durata della finestra gestuale
<code>FACE_COOLDOWN</code>	2.0 s	Attesa minima tra due attivazioni consecutive
<code>FACE_TOLERANCE</code>	0.50	Soglia distanza per il face matching (< = più severo)
<code>cv_scaler</code>	4	Fattore di riduzione del frame per la face detection

8. Installazione e Avvio

8.1 Prerequisiti

Il sistema richiede un ambiente virtuale Python con le dipendenze installate. I comandi per la configurazione iniziale sono i seguenti:

Setup ambiente:

```
python3 -m venv --system-site-packages face_rec
source face_rec/bin/activate
sudo apt update && sudo apt full-upgrade
pip install opencv-python imutils face-recognition mediapipe
sudo apt install cmake
```

Ogni volta che riapri il terminale:

```
cd && source face_rec/bin/activate && cd YourProjectFolder/
```

8.2 Procedura Operativa

La procedura completa per la messa in funzione del sistema si articola in tre fasi:

- **Passo 1 – Raccolta immagini:** Eseguire `image_capture.py` e fotografare gli utenti autorizzati, salvando le immagini nel dataset.
- **Passo 2 – Training:** Eseguire `model_training.py` per generare il file `encodings.pickle` con gli encoding facciali.
- **Passo 3 – Avvio sistema:** Eseguire lo script principale (`magic.py` per la versione multi-thread).

9. Considerazioni Finali e Sviluppi Futuri

MagicHands dimostra come sia possibile integrare tecnologie di visione artificiale avanzate su hardware embedded a basso costo come il Raspberry Pi 4. La combinazione di face recognition e gesture detection con MediaPipe permette di realizzare un'interfaccia uomo-macchina intuitiva, sicura e contactless.

Il confronto tra la versione sequenziale e quella con threading evidenzia chiaramente i vantaggi dell'approccio concorrente in termini di reattività e robustezza del sistema. La versione threading gestisce meglio i casi limite (attivazioni multiple, comandi lenti) pur mantenendo la semplicità logica della versione sequenziale come riferimento.

Possibili sviluppi futuri del progetto includono:

- Ampliamento della mappatura gestuale con gesti più complessi (ad es. scorrimento playlist)
- Aggiunta di un display OLED per mostrare il nome dell'utente riconosciuto e il gesto rilevato